

AI-Powered Enterprise E-Commerce Platform

Testing Strategy and Quality Assurance Plan

Document Information

Item	Description
Project Name	AI-Powered Enterprise E-Commerce Platform
Document Type	Testing Strategy
Version	1.0
Prepared By	Project Team
Status	Draft

Revision History

Version	Date	Description
1.0	DD/MM/YYYY	Initial Version

Table of Contents

- 1. Introduction
- 2. Purpose
- 3. Testing Objectives
- 4. Testing Principles
- 5. Testing Scope
- 6. Testing Levels
- 7. Testing Types
- 8. Test Environment
- 9. Test Data Management
- 10. Defect Management
- 11. Entry and Exit Criteria
- 12. Testing Metrics
- 13. Automation Strategy
- 14. AI Model Testing
- 15. Security Testing
- 16. Performance Testing

17. User Acceptance Testing
 18. Risks and Mitigation
 19. Conclusion
 20. References
-

1. Introduction

Testing is a critical activity that ensures software quality and reliability.

This document defines the testing strategy for the AI-Powered Enterprise E-Commerce Platform.

The purpose of testing is to verify that:

- Functional requirements are satisfied.
 - Non-functional requirements are achieved.
 - AI services produce acceptable results.
 - Security vulnerabilities are minimized.
 - System reliability is ensured.
-

2. Purpose

This document aims to:

- Define testing activities.
 - Establish quality standards.
 - Identify testing responsibilities.
 - Reduce software defects.
 - Improve system reliability.
-

3. Testing Objectives

The testing objectives are:

TO-01

Verify all functional requirements.

TO-02

Validate AI model outputs.

TO-03

Ensure system security.

TO-04

Evaluate system performance.

TO-05

Confirm integration between services.

TO-06

Improve overall software quality.

4. Testing Principles

Testing activities shall follow the following principles:

1. Testing shows the presence of defects, not their absence.
 2. Early testing reduces cost.
 3. Defects tend to cluster.
 4. Exhaustive testing is impossible.
 5. Testing activities should be risk-based.
 6. Testing should be repeatable and automated whenever possible.
-

5. Testing Scope

In Scope

Backend Services

- Authentication
- Product Service
- Cart Service
- Order Service
- Payment Service
- Inventory Service

AI Services

- Recommendation Engine
- Fraud Detection
- Forecasting Models

Frontend

- User Interfaces
- API Integration

- Validation

Infrastructure

- Docker Deployment
 - Database Connections
 - RabbitMQ Communication
-

Out of Scope

- Third-party internal systems.
 - Mobile applications.
 - Multi-region deployment testing.
-

6. Testing Levels

6.1 Unit Testing

Purpose:

Test individual components.

Examples:

- Service methods
- Utility functions
- Repository methods

Tools:

- JUnit 5
 - Mockito
 - PyTest
-

6.2 Integration Testing

Purpose:

Verify interaction between components.

Examples:

- Frontend ↔ Backend
- Spring Boot ↔ PostgreSQL
- Spring Boot ↔ FastAPI

- RabbitMQ Messaging
-

6.3 System Testing

Purpose:

Test the complete application.

Examples:

- User registration flow
 - Product purchasing flow
 - Fraud analysis flow
-

6.4 Acceptance Testing

Purpose:

Validate business requirements.

Performed by:

- Stakeholders
 - Project Team
-

7. Testing Types

Functional Testing

Validate:

- Business logic
 - User stories
 - Use cases
-

Regression Testing

Purpose:

Ensure new changes do not break existing functionality.

Smoke Testing

Purpose:

Verify critical functionality after deployments.

Sanity Testing

Purpose:

Validate bug fixes.

Exploratory Testing

Purpose:

Identify unexpected defects.

Compatibility Testing

Purpose:

Ensure functionality across browsers and devices.

Installation Testing

Purpose:

Verify Docker deployment and setup procedures.

Database Testing

Purpose:

Validate:

- Constraints
 - Relationships
 - Transactions
 - Stored procedures
-

API Testing

Purpose:

Verify:

- Endpoints
- Authentication
- Responses
- Error handling

Tools:

- Postman
 - Newman
-

8. Test Environment

Development Environment

Purpose:

Developer testing.

Staging Environment

Purpose:

Pre-production testing.

Production Environment

Purpose:

Final validation.

Environment Components

- Docker Containers
- PostgreSQL
- Redis
- RabbitMQ
- Spring Boot Services
- FastAPI Services

9. Test Data Management

Test data should include:

User Data

- Valid users
- Invalid users
- Different roles

Product Data

- Small datasets
- Large datasets
- Invalid products

Transaction Data

- Successful payments
- Failed payments
- Fraud scenarios

AI Datasets

- Historical sales data
 - Recommendation datasets
 - Fraud datasets
-

10. Defect Management Process

```
graph TD; A[Identify Defect] --> B[Log Defect]; B --> C[Prioritize]; C --> D[Fix]; D --> E[Retest]; E --> F[Close];
```

Defect Severity Levels

Level	Description
Critical	System unusable
High	Major functionality failure
Medium	Partial functionality issue
Low	Cosmetic issue

Defect Priority Levels

Priority	Description
P1	Immediate Fix
P2	High Priority
P3	Medium Priority
P4	Low Priority

11. Entry Criteria

Testing may begin when:

- Requirements are approved.
 - Code is deployed.
 - Test environment is available.
 - Test cases are prepared.
-

12. Exit Criteria

Testing is considered complete when:

- Critical defects = 0
 - High defects are resolved
 - Test coverage targets achieved
 - Stakeholder approval obtained
-

13. Test Coverage Targets

Category	Target
Backend	>80%
AI Services	>75%
Frontend	>70%
Critical APIs	>90%

14. Testing Metrics

Examples:

Defect Density

Defects / KLOC

Test Coverage

Executed Tests / Total Tests

Defect Leakage

Production Defects / Total Defects

Mean Time to Repair (MTTR)

Measure defect resolution efficiency.

15. Automation Testing Strategy

Backend Automation

Tools:

- JUnit

- Mockito
-

Python Automation

Tools:

- PyTest
-

API Automation

Tools:

- Postman
 - Newman
-

Frontend Automation

Tools:

- React Testing Library
 - Cypress
-

CI Testing

Future:

- GitHub Actions
 - Jenkins
-

16. Performance Testing

Purpose:

Evaluate scalability and responsiveness.

Test Scenarios

User Login

Target:

< 2 seconds.

Product Search

Target:

< 3 seconds.

Recommendation Service

Target:

< 5 seconds.

Concurrent Users

Target:

1,000+ users.

Performance Tools

- JMeter
 - k6
-

17. Security Testing

Purpose:

Identify vulnerabilities.

Authentication Testing

Verify:

- JWT
 - Session management
 - RBAC
-

OWASP Testing

Validate protection against:

- SQL Injection
 - XSS
 - CSRF
 - Broken Authentication
 - Sensitive Data Exposure
-

Security Testing Tools

- OWASP ZAP
 - SonarQube
-

18. AI Model Testing

Testing AI components requires additional validation.

Recommendation System Testing

Metrics:

- Precision@K
 - Recall@K
 - MAP
 - NDCG
-

Fraud Detection Testing

Metrics:

- Accuracy
 - Precision
 - Recall
 - F1 Score
 - ROC-AUC
-

Forecasting Testing

Metrics:

- MAE
 - RMSE
 - MAPE
-

Bias Testing

Ensure:

- Fair recommendations

- Balanced predictions
 - No severe model bias
-

Model Drift Testing

Monitor:

- Performance degradation over time.
-

19. User Acceptance Testing (UAT)

Purpose:

Validate business expectations.

Example UAT Scenarios

Customer Journey

- Registration
 - Product Search
 - Add to Cart
 - Purchase Product
-

Administrator Journey

- Add Product
 - Manage Inventory
 - View Reports
-

Business Owner Journey

- View Analytics
 - Review Forecasts
-

Acceptance Criteria

Users should successfully complete major workflows.

20. Test Deliverables

Documents

- Test Plan
 - Test Cases
 - Defect Reports
 - Test Summary Reports
-

Artifacts

- Automated Test Scripts
 - Coverage Reports
 - Performance Reports
 - Security Reports
-

21. Risks and Mitigation

Risk	Mitigation
Limited test data	Generate synthetic data
Environment instability	Docker environments
AI model variability	Multiple evaluations
Schedule delays	Incremental testing

22. Roles and Responsibilities

Role	Responsibilities
Developers	Unit Testing
QA Engineers	System Testing
AI Engineers	AI Validation
Project Manager	Quality Monitoring

23. Future Enhancements

Possible future improvements:

- Chaos Engineering
- Mutation Testing
- Contract Testing

- Kubernetes Testing
 - A/B Testing for Recommendations
 - Continuous Performance Testing
-

24. Conclusion

The testing strategy ensures that the AI-Powered Enterprise E-Commerce Platform achieves high levels of:

- quality,
- reliability,
- security,
- maintainability,
- and performance.

The combination of traditional software testing and AI-specific testing practices provides comprehensive quality assurance for the entire platform.

References

- [1] ISTQB Foundation Level Syllabus.
- [2] Sommerville, I. (2016). Software Engineering (10th Edition).
- [3] Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach.
- [4] Myers, G. J., Sandler, C., & Badgett, T. (2011). The Art of Software Testing.
- [5] OWASP Testing Guide.
- [6] JUnit Documentation.
- [7] PyTest Documentation.
- [8] Cypress Documentation.